

Lec: Prefix — Practice

Qul

A[]

Medium

Q queries — l & r

for each query. return sum of all odd indexed elements.

0	1	2	3	4	5
2	3	1	6	4	5

1	3	— $A[1] + A[3] = 9$
2	5	— $A[3] + A[5] = 11$
0	5	— $A[1] + A[3] + A[5] = 14$
2	2	— 0

Brute force:

```
fun querysum(A[], q[l][r]) {  
    q = q.length;  
    for(i=0 ————— q-1)  
        l = q[i][0];  
        r = q[i][1];  
        sum = 0;  
        for(j=l ————— r) {  
            if(j%2 == 1) {  
                sum += A[j]  
            }  
        }  
        print(sum);  
    }  
}
```

TC: $O(q * n)$

SC: $O(1)$

	0	1	2	3	4	5	6	7	8	9
A =	-3	6	2	4	5	2	8	-9	3	1
pf[n] →	0	6	6	10	10	12	12	3	3	4

Diagram showing the calculation of prefix sums for odd indices. Red arrows point from pf[0] to pf[1] and from pf[1] to pf[2]. Blue arrows point from pf[2] to pf[3], pf[3] to pf[4], pf[4] to pf[5], pf[5] to pf[6], and pf[6] to pf[7].

$pf[i] \Rightarrow$ sum of all odd indexed elements from 0th to i th idx.

$$\begin{aligned}
 pf[0] &= 0 \\
 pf[1] &= pf[0] + 1 \% 2 \cdot A[1] \\
 &= pf[0] \\
 pf[2] &= pf[1] + 2 \% 2 \cdot A[2]
 \end{aligned}$$

Diagram illustrating the calculation of prefix sums for odd indices. For $pf[1]$, the value is $pf[0] + 1 \% 2 \cdot A[1]$, resulting in $0 + 0 \cdot 6 = 0$. For $pf[2]$, the value is $pf[1] + 2 \% 2 \cdot A[2]$, resulting in $0 + 0 \cdot 2 = 0$.

```

int[] getOddIndicesPrefixsum(A[]) {
    pf[n];
    pf[0] = 0;
    for (i = 1; i < n; i++) {
        if (i % 2 == 1) {
            pf[i] = pf[i-1] + A[i];
        } else {
            pf[i] = pf[i-1];
        }
    }
    return pf;
}

```

TC: $O(n)$
 SC: $O(n)$
 ✓

$O(1)$

```

fun querysum(A[], q[l][r]) {
    O(n) _____ pf[] = getIndicesPrefixsum(A);
                    q = q.length;
    O(q) _____ for(i=0 _____ q-1) {
        l = q[i][0];
        r = q[i][1];
        if(l == 0) {
            sum = pf[r];
        } else {
            sum = pf[r] - pf[l-1];
        }
        print(sum);
    }
}
TC: O(q+n)
SC: O(n)

```

pf:

0	1	2	3	4
2	4	3	1	5
0	4	4	5	5

Qu

A[n]

Amazon

Count special indices in an array.Med-hard

index after removing,
 sum of all even indexed el = sum of all odd indexed elements

0	1	2	3	4	5
4	3	2	7	6	-2

Ans=2

i	updated array.	special?
0	⁰ 3 ¹ 2 ² 7 ³ 6 ⁴ -2	$S_e = 8$ $S_o = 8$ ✓
1	⁰ 4 ¹ 2 ² 7 ³ 6 ⁴ -2	$S_e = 9$ $S_o = 8$ ✗
2	⁰ 4 ¹ 3 ² 7 ³ 6 ⁴ -2	$S_e = 9$ $S_o = 9$ ✓
...		

0	1	2	3	4
4	1	3	7	10

Qu sum of elements at odd indices after removing idx=2
⁰4 ¹1 ²7 ³10 $\longrightarrow S_o = 11$

0	1	2	3	4	5	6	7	8	9
2	3	1	4	0	-1	2	-2	10	8

Qu sum of elements at odd indices after removing idx=3
⁰2 ¹3 ²1 ³0 ⁴-1 ⁵2 ⁶-2 ⁷10 ⁸8 $\longrightarrow S_o = 15$

0	1	2	3	4	5	6	7	8	9
2	3	1	4	0	-1	2	-2	10	3

Qu sum of elements at even. indices after removing $idx=3$

$$\begin{array}{cccccccc} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 2 & 3 & 1 & 0 & -1 & 2 & -2 & 10 & 8 \end{array} \rightarrow a_c = 8 \text{ Ans}$$

Observations

0	1	2	3	4	5	6	7	8	9
2	3	1	4	6	-1	2	-2	10	8

Remove $\text{idx} = 3$.

0	1	2	3	4	5	6	7	8
2	3	1	0	-1	2	-2	10	8

After $idx = l$ is removed

$[i+1, n-1]$ ——— even indices become odd indices
odd " " even "

$[0, x^0-1]$ ————— No change

Sum of odd indexed el after removing $idx = 3$

$$\begin{array}{lcl} \text{sum}_{\text{odd}} [0, 2] & + & \text{sum}_{\text{even}} [4, 9] \\ [3] & + & [0 + 2 + 10] \longrightarrow \underline{15 \text{ Ans}} \end{array}$$

Sum of even. indexed el after removing $idx = 3$

$$\begin{array}{l} \text{sum}_{\text{even}} [0, 2] + \text{sum}_{\text{odd}} [4, 9] \\ [2, 1] + [-1, -2, +8] = \underline{8} \text{ Ans} \end{array}$$

pfOdd[]
pfEven[]

Sum of odd indexed el after removing i^{th} = i



$$\text{Ans} = \text{sum}_{\text{odd}}[0, i-1] + \text{sum}_{\text{even}}[i+1, n-1]$$



$$\begin{aligned} & \text{pf}_{\text{odd}}[i-1] + \text{pf}_{\text{even}}[n-1] - \text{pf}_{\text{even}}[i+1] \\ & \text{pf}_{\text{odd}}[i-1] + \text{pf}_{\text{even}}[n-1] - \text{pf}_{\text{even}}[i] \end{aligned}$$

Edge case: $i = 0$

$$\text{Ans} = \text{pf}_{\text{even}}[n-1] - \text{pf}_{\text{even}}[i]$$

Sum of even indexed el after removing i^{th} = i



$$\text{Ans} = \text{sum}_{\text{even}}[0, i-1] + \text{sum}_{\text{odd}}[i+1, n-1]$$



$$\begin{aligned} & \text{pf}_{\text{even}}[i-1] + \text{pf}_{\text{odd}}[n-1] - \text{pf}_{\text{odd}}[i+1] \\ & \text{pf}_{\text{even}}[i-1] + \text{pf}_{\text{odd}}[n-1] - \text{pf}_{\text{odd}}[i] \end{aligned}$$

Edge case: $i = 0$

$$\text{Ans} = \text{pf}_{\text{odd}}[n-1] - \text{pf}_{\text{odd}}[i]$$

function countSpecialIndices (A[]) {

pfOdd = getOddIndicesPrefixsum (A); $\text{--- } O(n), O(n)$

pfEven = getEvenIndicesPrefixsum (A); $\text{--- } O(n), O(n)$

count = 0;

$O(n)$ --- for (i = 0 --- n-1) {

if (i == 0) {

so = pfEven[n-1] - pfEven[i];

se = pfOdd[n-1] - pfOdd[i];

} else {

so = pfOdd[i-1] + pfEven[n-1] - pfEven[i];

se = pfEven[i-1] + pfOdd[n-1] - pfOdd[i];

}

if (so == se) {

count++;

}

}

return count;

}

TC: $n + n + n \rightarrow O(n)$

sc: $O(n)$

Qu

A[]

Medium

Q queries ——— l and r.

for each query. return sum of all even elements from l to r.

0	1	2	3	4	5
2	3	1	6	4	5

1	3	Ans = 6
2	5	Ans = 10
0	5	2 + 6 + 4 = 12 Ans
2	2	0

Brute force:

```

fun querysum(A[], q[][[]]) {
    q = q.length;
    for(i=0 ————— q-1)
        l = q[i][0];
        r = q[i][1];
        sum = 0;
        for(j=l ————— r) {
            if (A[j] % 2 == 0) {
                sum += A[j];
            }
        }
        print(sum);
    }
}

```

TC: $O(q * n)$ SC: $O(1)$

	0	1	2	3	4	5	6	7	8	9
A =	-3	6	2	4	5	2	8	-9	3	1
pf[n] →	0	6	8	12	12	14	22	22	22	22
		↑	↑	↑	↑	↑	↑	↑	↑	↑

pf[i] ⇒ sum of all even. elements from 0th to ith idx.

$$\begin{aligned}
 pf[0] &= A[0] \% 2 == 0 ? A[0] : 0; \\
 pf[1] &= pf[0] + A[1] \% 2 \begin{cases} 0 & A[1] \\ 1 & 0 \checkmark \end{cases} \\
 &= pf[0] \\
 pf[2] &= pf[1] + A[2] \% 2 \begin{cases} 0 & A[2] \checkmark \\ 1 & 0 \end{cases}
 \end{aligned}$$

```

int[] getEvenElementsPrefixsum(A[]) {
    pf[n];
    pf[0] = A[0] \% 2 == 0 ? A[0] : 0;
    for(i=1; i < n; i++) {
        if (A[i] \% 2 == 0) {
            pf[i] = pf[i-1] + A[i];
        } else {
            pf[i] = pf[i-1];
        }
    }
    return pf;
}

```

TC: $O(n)$

SC: $O(n)$ ✓

$O(1)$

```

fun querysum(A[], Q[][]) {
  O(n) _____ pf[] = getEvenElementsPrefixsum(A);
                    q = Q.length;
  O(q) _____ for(i=0 _____ q-1) {
                    l = Q[i][0];
                    r = Q[i][1];
                    if(l == 0) {
                      sum = pf[r];
                    } else {
                      sum = pf[r] - pf[l-1];
                    }
                    print(sum);
                }
}

```

TC: $O(q+n)$
 sc: $O(n)$

Break: 10:22 - 10:30

Q.1

A[]

find size of smallest subarray that contains at least one occurrence of min & max value of A[]

1	3	2
---	---	---

Min = 1
Max = 3

3	1	4	2	5	3	2	4	2	1
---	---	---	---	---	---	---	---	---	---

min = 1
max = 5

Brute force:

Go to all subarrays

for each subarray

check if it has max & min

yes — updating ans.

min = getMin(A); $O(n)$

max = getMax(A); $O(n)$

ans = ∞ ;

for (i = 0; i < n; i++) {

for (j = i; j < n; j++) {

for (k = i; k < j; k++) {

check if it has max & min

yes — updating ans.

}

}

}

Approach

0	1	2	3	4	5	6	7	8	9
3	1	4	2	5	3	2	4	2	1

$$\min = 1 \quad \text{—————} \quad O(n)$$

$$\max = 5 \quad \text{—————} \quad O(n)$$

$$\min_idx = -1;$$

$$\max_idx = -1;$$

$$\text{ans} = \infty;$$

$$i=0 \quad \text{—————}$$

$$i=1 \quad \text{—————} \quad \min_idx = 1.$$

$$i=2 \quad \text{—————}$$

$$i=3 \quad \text{—————}$$

$$i=4 \quad \text{—————} \quad \left. \begin{array}{l} \max_idx = 4 \\ \min_idx = 1 \end{array} \right\} \text{—————} \quad \begin{array}{l} \text{size} = 4 - 1 + 1 = 4 \\ \text{ans} = 4 \end{array}$$

$$i=5 \quad \text{—————} \quad \left. \begin{array}{l} \max_idx = 4 \\ \min_idx = 1 \end{array} \right\} \text{—————} \quad \begin{array}{l} \text{size} = 4 - 1 + 1 = 4 \\ \text{ans} = 4 \end{array}$$

$$i=6 \quad \text{—————} \quad \text{ans} = 4$$

$$i=7 \quad \text{—————} \quad \text{ans} = 4$$

$$i=8 \quad \text{—————} \quad \text{ans} = 4$$

$$i=9 \quad \text{—————} \quad \left. \begin{array}{l} \min_idx = 9 \\ \max_idx = 4 \end{array} \right\} \text{—————} \quad \begin{array}{l} \text{size} = 9 - 4 + 1 = 5 \\ \underline{\underline{\text{ans} = 4}} \end{array}$$

fun getSmallestSubArray (A[]) {

min = $+\infty$; Integer.MAX-VALUE
max = $-\infty$;

for (i = 0 _____ n-1) {

O(n) _____

min = Math.min(min, A[i]);

max = Math.max(max, A[i]);

}

min-idx = -1;

max-idx = -1;

ans = $+\infty$;

if (min == max) {

return 1; _____ All el are same

}

for (i = 0 _____ n-1) {

if (A[i] == min) {

min-idx = i;

}

if (A[i] == max) {

max-idx = i;

}

if (min-idx != -1 && max-idx != -1) {

size = Math.abs(min-idx - max-idx) + 1;

ans = min(ans, size);

}

}

return ans;

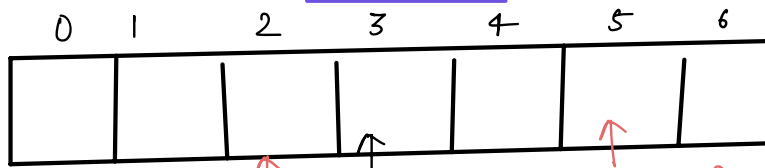
}

TC: O(n)

sc: O(1)

Thankyou 😊

Doubt



$$\text{sum}[0, 2] = \text{sum}[3, 6]$$

$$\text{sum}[0, 3] = \text{sum}[4, 6]$$

$$\text{sum}[0, 5] = \text{sum}[6, 6]$$